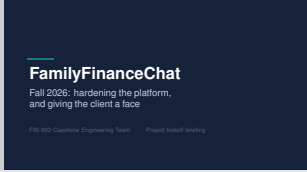
FamilyFinanceChat

Fall 2026: hardening the platform, and giving the client a face

FIN 602 Capstone Engineering Team · Project kickoff briefing



Welcome. Two workstreams this semester: finish making the platform professional, and investigate embodied avatars. Set the tone – this is a real production system with real users.

What we will cover

Part 1 — The platform

1. Where the system stands today
2. The lesson that shaped it: the fork
3. The compatibility contract
4. What we still have to strip out
5. Closing the version gap
6. Making deployment real

Part 2 — The new bet

7. Why typing is not enough
8. Avatars: the target experience
9. Architecture, latency, and cost
10. How we will evaluate it honestly
11. Timeline, roles, and risks

The one rule that governs everything

We never modify Open WebUI's core. If a feature cannot be built on a published extension surface, the feature changes — not the core.

Two halves. Flag the rule early – it recurs in every architectural decision, including the avatar design.

The problem this platform solves

For students

FIN 602 students must practise client-facing financial advising before they sit across from a real family.

Human role-players do not scale: scheduling bottlenecks, inconsistent scenarios, uneven feedback.

For instructors

Reading every transcript by hand does not scale either.

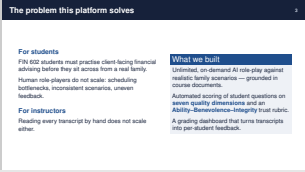
What we built

Unlimited, on-demand AI role-play against realistic family scenarios — grounded in course documents.

Automated scoring of student questions on **seven quality dimensions** and an **Ability–Benevolence–Integrity** trust rubric.

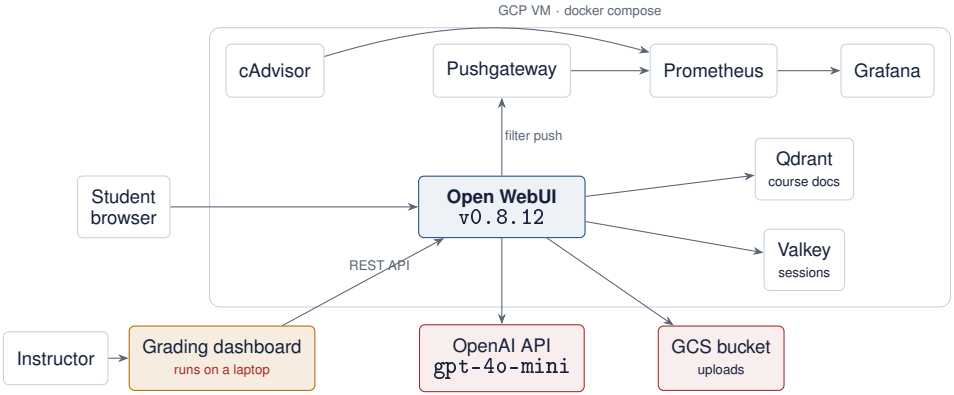
A grading dashboard that turns transcripts into per-student feedback.

The problem this platform solves



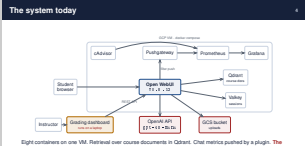
Ground the audience in purpose before architecture. Scale + consistency is the value proposition. ABI is the pedagogical differentiator.

The system today



Eight containers on one VM. Retrieval over course documents in Qdrant. Chat metrics pushed by a plugin. **The grading dashboard is the outlier — it is not hosted.**

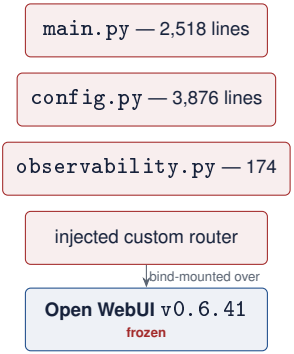
└─The system today



Walk the diagram left to right. Land on the one thing in amber: the professor's tool runs on a laptop. That is workstream A's headline problem.

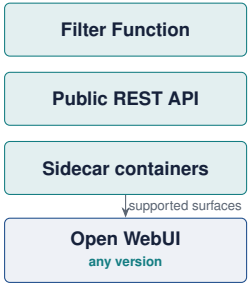
The lesson that shaped this project

Before — Spring 2026 and earlier



≈6,500 lines of forked vendor code, re-merged by hand on every upgrade. Users reached a feature via a *browser bookmarklet*.

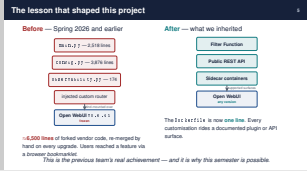
After — what we inherited



The `Dockerfile` is now **one line**. Every customisation rides a documented plugin or API surface.

This is the previous team's real achievement — and it is why this semester is possible.

The lesson that shaped this project



Give the last team credit. Six and a half thousand lines of fork to get two features. The bookmarklet detail always lands – mention it.

Where it still hurts

Problem	Consequence
Pinned to `v0.8.12`; upstream ships `v0.11.x`	Three minor versions of fixes, features — and *migrations* — deferred
Metrics plugin is pasted into the admin UI by hand after every deploy	Deployment is not reproducible; metrics vanish silently
That plugin is synchronous, with blocking network I/O and shared state	On the v0.9+ async core it degrades every concurrent user
Grading dashboard is not hosted	A professor needs SSH and a terminal to grade
No CI, no automated smoke test	Past upgrades broke things silently
No alerting on `/ready`	An outage was discovered from students
Two RAG stacks, two scoring implementations	Ambiguity about which numbers are canonical
Default `admin`/`admin`; unrotated leaked key	Straightforward security debt

None of this is exotic. It is the gap between *a system that works* and *a system you can hand over*.

Where it still hurts

Problem	Consequence
Pinned to `rust:1.70` ; upstream ships `rust:1.72`	Three minor versions of fixes, features — and migrations — deferred
Metrics plugin is pasted into the admin UI by hand after every deploy	Deployment is not reproducible; metrics vanish silently
That plugin is synchronous, with blocking network calls in request paths	On the v0.9+ async core it degrades every concurrent request
Grading dashboard is not hosted	A professor needs SSH and a terminal to grade
No CI, no automated smoke test	Past upgrades broke things silently
No alerting on `/readyz`	An outage was discovered from students
Two RAG stacks, two scoring implementations	Ambiguity about which numbers are canonical
Default `admin/admin` , unrotated leaked key	Straightforward security debt

None of this is exotic. It is the gap between *a system that works* and *a system you can hand over*.

Read only three or four rows aloud – hosted dashboard, no CI, version gap. Close with the "gap between works and handover" line.

Workstream A

Harden and modernise

Weeks 1–15 · this must not slip

The compatibility contract

The rule

Open WebUI is a **dependency**, not a codebase we own. We must be able to pull any published release and have the platform work — with no merge, no patch, no re-fork.

Every feature must ride a surface that upstream **publishes, documents, and versions**.

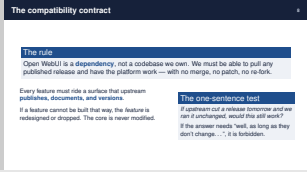
If a feature cannot be built that way, the *feature* is redesigned or dropped. The core is never modified.

The one-sentence test

If upstream cut a release tomorrow and we ran it unchanged, would this still work?

If the answer needs “well, as long as they don’t change. . . ”, it is forbidden.

The compatibility contract

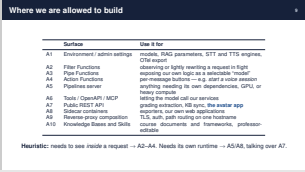


Slow down here. This is the intellectual core of the deck. The one-sentence test is what they should apply in code review all semester.

Where we are allowed to build

	Surface	Use it for
A1	Environment / admin settings	models, RAG parameters, STT and TTS engines, OTel export
A2	Filter Functions	observing or lightly rewriting a request in flight
A3	Pipe Functions	exposing our own logic as a selectable “model”
A4	Action Functions	per-message buttons — e.g. *start a voice session*
A5	Pipelines server	anything needing its own dependencies, GPU, or heavy compute
A6	Tools / OpenAPI / MCP	letting the model call our services
A7	Public REST API	grading extraction, KB sync, **the avatar app**
A8	Sidecar containers	exporters, our own web applications
A9	Reverse-proxy composition	TLS, auth, path routing on one hostname
A10	Knowledge Bases and Skills	course documents and frameworks, professor-editable

Heuristic: needs to see *inside* a request → A2–A4. Needs its own runtime → A5/A8, talking over A7.

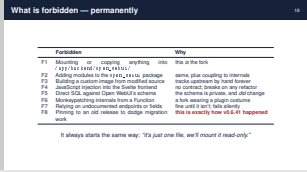


Do not read the table. Point at A7 and A8 – those two are what the avatar work will use. The heuristic line is the takeaway.

What is forbidden — permanently

	Forbidden	Why
F1	Mounting or copying anything into `/app/backend/open_webui/`	this *is* the fork
F2	Adding modules to the `open_webui` package	same, plus coupling to internals
F3	Building a custom image from modified source	tracks upstream by hand forever
F4	JavaScript injection into the Svelte frontend	no contract; breaks on any refactor
F5	Direct SQL against Open WebUI's schema	the schema is private, and *did* change
F6	Monkeypatching internals from a Function	a fork wearing a plugin costume
F7	Relying on undocumented endpoints or fields	fine until it isn't; fails silently
F8	Pinning to an old release to dodge migration work	**this is exactly how v0.6.41 happened**

It always starts the same way: *"it's just one file, we'll mount it read-only."*



F6 and F8 are the ones teams actually violate. The closing quote is the warning to remember.

Audit: what we still have to strip out

Good news: the active stack is already compliant — no compose file mounts anything into Open WebUI's source tree. What remains is peripheral coupling and *loaded guns left in the tree.*

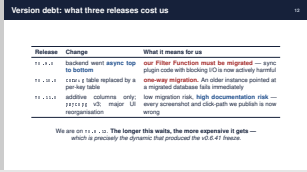
	Finding	Action	Risk
C-1	the vendor-fork source under `legacy/` — with a README telling you how to re-fork	delete from the tree; leave a tombstone pointing at the commit	**High**
C-2	`-legacy` direct-SQLite flag (a `TODO` stub)	delete the flag; REST API is the only path	Med
C-3	custom `build:` context for Open WebUI	replace with a pinned `image:`; delete the `Dockerfile`	Low
C-4	metrics filter: sync, blocking I/O, shared state	rewrite async, or retire it for native OTel	**High**
C-6	`:latest` images (Qdrant, cAdvisor)	pin every image	Med
C-7	Grafana 9.0.0, default `admin`/`admin`	upgrade; remove the default password	**High**
C-8/9	two RAG stacks, two scoring implementations	pick one canonical each, in writing	Med
C-10	previously leaked provider key not yet rotated	rotate at the provider; add `gitleaks` to CI	**High**

C-1 is the headline: the repo still contains working instructions for re-creating the fork. Deleting it is a five-minute job with outsized value.

Version debt: what three releases cost us

Release	Change	What it means for us
`v0.9.0`	backend went **async top to bottom**	**our Filter Function must be migrated** — sync plugin code with blocking I/O is now actively harmful
`v0.10.0`	`config` table replaced by a per-key table	**one-way migration.** An older instance pointed at a migrated database fails immediately
`v0.11.0`	additive columns only; `psycopg` v3; major UI reorganisation	low migration risk, **high documentation risk** — every screenshot and click-path we publish is now wrong

We are on `v0.8.12`. **The longer this waits, the more expensive it gets —**
which is precisely the dynamic that produced the v0.6.41 freeze.



The point is not "upgrades are nice". It is that deferred migrations compound, and this project already has the scar to prove it.

How we will do the upgrade

1. Snapshot production — and **restore the snapshot into the test stack**. *A backup you have never restored is not a backup.*
2. Upgrade the test stack first, on real production data.
3. Move **one minor version at a time**: 0.8 → 0.9 → 0.10 → 0.11. Migrations run sequentially; a single jump makes failures unattributable.
4. Run the smoke suite at every step. Write the runbook *as you go*.
5. Decide **SQLite → PostgreSQL** explicitly, and record the decision either way.
6. Schedule the production window with the rollback written *before* you start.
7. Re-shoot every screenshot for the new UI.

Done when

Production runs the current release.

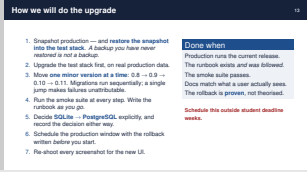
The runbook exists *and was followed*.

The smoke suite passes.

Docs match what a user actually sees.

The rollback is **proven**, not theorised.

Schedule this outside student deadline weeks.



Step 1 and step 6 are the professional habits worth arguing for. Most teams skip both and get away with it until they don’t.

CI/CD — and the guard that keeps the fork dead

Pipeline, by value per hour spent

1. **Smoke test on every push** — bring the stack up, wait for `/health` then `/ready`, send a chat completion, upload a document, assert retrieval returns it
2. **Fork guard** (below)
3. **Secret scanning** — `gitleaks`, including history
4. **Lint and unit tests** — especially on the scoring service: *it produces numbers that become grades*
5. **Deploy on tag** — pull, restart, re-smoke, auto-rollback

The fork guard — fails the build when. . .

a compose file mounts into `open_webui/`

the image is built from modified source

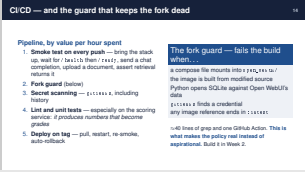
Python opens SQLite against Open WebUI's data

`gitleaks` finds a credential

any image reference ends in `:latest`

≈40 lines of grep and one GitHub Action. **This is what makes the policy real instead of aspirational.** Build it in Week 2.

└CI/CD — and the guard that keeps the fork dead



Emphasise: a policy nobody enforces is a wish. Forty lines of grep converts a document into a constraint.

Two fixes that change who can use this

Host the grading dashboard

Today a professor must SSH into a GCP VM and run a shell script.

Containerise the backend, build the frontend to static assets, put Nginx in front with TLS and real authentication, and give them a **URL**.

Watch: make `/refresh` asynchronous with visible progress — it scores every question through a paid API.

Done when a professor grades with no terminal, no VPN, and no help from a student.

Observability without manual steps

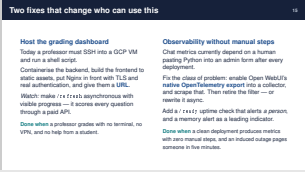
Chat metrics currently depend on a human pasting Python into an admin form after every deployment.

Fix the *class* of problem: enable Open WebUI's **native OpenTelemetry export** into a collector, and scrape that. Then retire the filter — or rewrite it async.

Add a `/ready` uptime check that alerts *a person*, and a memory alert as a leading indicator.

Done when a clean deployment produces metrics with zero manual steps, and an induced outage pages someone in five minutes.

Two fixes that change who can use this



These two are the highest-adoption-value items in the whole plan. The dashboard one is the difference between a tool that gets used and one that doesn't.

Remove the ambiguity

Scoring exists twice

JavaScript and Python, kept in sync by hand.

Declare **Python canonical**. Retire the prototype. Add tests that pin the rubric arithmetic and ABI weights.

RAG exists twice

Native Qdrant (what students use) and a standalone ChromaDB pipeline (which nothing uses).

Retire it, or promote it behind a **Pipelines server**. Decide in writing.

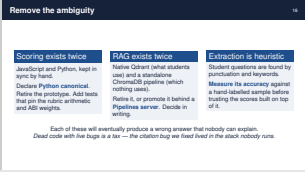
Extraction is heuristic

Student questions are found by punctuation and keywords.

Measure its accuracy against a hand-labelled sample before trusting the scores built on top of it.

Each of these will eventually produce a wrong answer that nobody can explain.

Dead code with live bugs is a tax — the citation bug we fixed lived in the stack nobody runs.



The third column is the sleeper. If question extraction is 70 percent accurate, every grade downstream inherits that error, silently.

Workstream B

Embodied advising

The new research track · Weeks 3–15

Why typing is not enough

FIN 602 exists to prepare students for **a room with a real family in it**.

That room demands: listening while someone is still talking, hearing hesitation, holding eye contact through an uncomfortable question about money, and asking the follow-up *out loud* — without a backspace key.

The gap

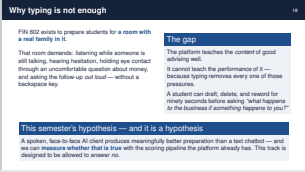
The platform teaches the *content* of good advising well.

It cannot teach the *performance* of it — because typing removes every one of those pressures.

A student can draft, delete, and reword for ninety seconds before asking *"what happens to the business if something happens to you?"*

This semester's hypothesis — and it is a hypothesis

A spoken, face-to-face AI client produces meaningfully better preparation than a text chatbot — and we can **measure whether that is true** with the scoring pipeline the platform already has. This track is designed to be allowed to answer *no*.



This is the emotional centre of the pitch. Deliver the backspace-key line slowly. Then immediately frame it as testable, not assumed.

What we are actually building

A student opens a link, sees a face, and says out loud:

> *"Hi — I'm going to ask you a few questions about your family's finances. Is that alright?"*

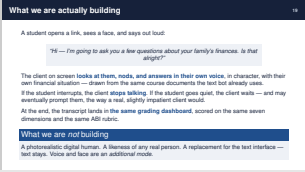
The client on screen **looks at them, nods, and answers in their own voice**, in character, with their own financial situation — drawn from the same course documents the text bot already uses.

If the student interrupts, the client **stops talking**. If the student goes quiet, the client waits — and may eventually prompt them, the way a real, slightly impatient client would.

At the end, the transcript lands in **the same grading dashboard**, scored on the same seven dimensions and the same ABI rubric.

What we are *not* building

A photorealistic digital human. A likeness of any real person. A replacement for the text interface — text stays. Voice and face are an *additional mode*.



Read the quoted line as if speaking to the avatar. The "stops talking" detail is what separates a real prototype from a demo video.

The constraint dictates the architecture

The instinct is to put the avatar *inside* the Open WebUI chat window.

That is forbidden — F3 and F4 at once

Open WebUI's frontend is a Svelte application we do not own. Embedding a WebRTC video pane in it means forking and re-merging that frontend forever — **precisely the mistake this project spent a semester undoing.**

So the constraint decides the design, and the design it produces is **better**:

The avatar is a companion web application in its own container.

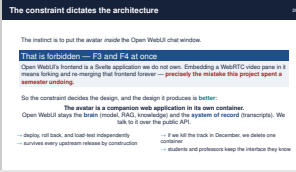
Open WebUI stays the **brain** (model, RAG, knowledge) and the **system of record** (transcripts). We talk to it over the public API.

→ deploy, roll back, and load-test independently

→ survives every upstream release by construction

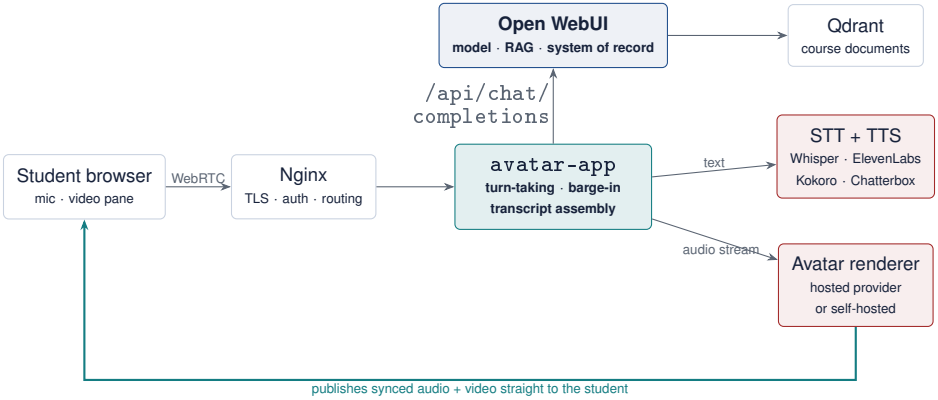
→ if we kill the track in December, we delete one container

→ students and professors keep the interface they know

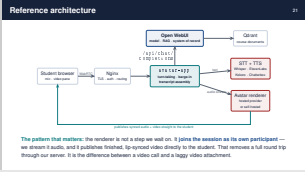


This slide is the argument that the compatibility policy is not bureaucracy – it produced a better architecture than the instinctive one.

Reference architecture



The pattern that matters: the renderer is not a step we wait on. It **joins the session as its own participant** — we stream it audio, and it publishes finished, lip-synced video directly to the student. That removes a full round trip through our server. It is the difference between a video call and a laggy video attachment.



Trace the teal arrow. Do not hand-roll WebRTC and turn-taking – use LiveKit Agents or Pipecat. That is not the semester's research question.

Latency is the number that decides everything

Stage	Realistic
Voice activity detection / endpointing	150–300 ms
Speech to text	150–300 ms
Retrieval (RAG)	100–300 ms
LLM time-to-first-token	300–600 ms
Text to speech, first byte	150–300 ms
Avatar render + network	200–500 ms
First audible syllable	**1.0–2.3 s**

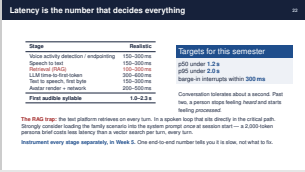
Targets for this semester

p50 under **1.2 s**
p95 under **2.0 s**
barge-in interrupts within **300 ms**

Conversation tolerates about a second. Past two, a person stops feeling *heard* and starts feeling *processed*.

The RAG trap: the text platform retrieves on every turn. In a spoken loop that sits directly in the critical path. Strongly consider loading the family scenario into the system prompt *once* at session start — a 2,000-token persona brief costs less latency than a vector search per turn, every turn.

Instrument every stage separately, in Week 5. One end-to-end number tells you it is slow, not what to fix.



The RAG trap is the most useful engineering insight on this slide – it is counterintuitive and it is where a naive implementation loses half a second per turn.

Three strategies — and the one we start with

1. Hosted streaming

HeyGen LiveAvatar, Tavus (Phoenix-4), Simli, Anam, bitHuman, Beyond Presence, D-ID.

Plug into LiveKit or Pipecat; swap providers behind one interface.

Vendor latency claims are measured under vendor conditions. Measuring ours is the assignment.

2. Self-hosted open source

Whisper → open TTS → an open lip-sync renderer.

For: no per-minute cost, no student audio leaving our infrastructure.

Against: needs a GPU we do not have; quality and latency lag; it becomes an infrastructure project.

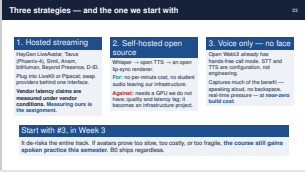
3. Voice only — no face

Open WebUI *already has* hands-free call mode. STT and TTS are configuration, not engineering.

Captures much of the benefit — speaking aloud, no backspace, real-time pressure — at **near-zero build cost**.

Start with #3, in Week 3

It de-risks the entire track. If avatars prove too slow, too costly, or too fragile, **the course still gains spoken practice this semester.** B0 ships regardless.



The strategic point: build the fallback first, and build it in three weeks. That is how you make an ambitious research track safe to attempt.

What it costs — and how we stop it running away

40 students $\times$ 4 sessions $\times$ 15 min $=$ **40 hours / semester**

Line item	Semester
Avatar streaming ($\sim$$0.10–0.20/min)	$240–480
Speech to text	$10–30
Text to speech	$20–60
LLM (short spoken turns)	$15–40
Total	$\approx$**$285–610**

Checked August 2026. Prices move quarterly — re-derive at signup and record the date.

Controls from day one — not after the first bill

15-minute hard cap, enforced server-side

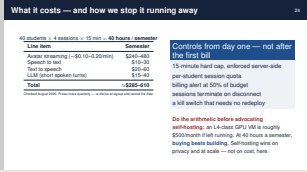
per-student session quota

billing alert at 50% of budget

sessions terminate on disconnect

a kill switch that needs no redeploy

Do the arithmetic before advocating self-hosting: an L4-class GPU VM is roughly $500/month if left running. At 40 hours a semester, **buying beats building.** Self-hosting wins on privacy and at scale — not on cost, here.



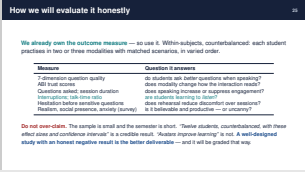
The self-hosting arithmetic is the slide's punchline. Students instinctively assume open source is cheaper; at this volume it is roughly ten times more expensive.

How we will evaluate it honestly

We already own the outcome measure — so use it. Within-subjects, counterbalanced: each student practises in two or three modalities with matched scenarios, in varied order.

Measure	Question it answers
7-dimension question quality	do students ask *better* questions when speaking?
ABI trust scores	does modality change how the interaction reads?
Questions asked; session duration	does speaking increase or suppress engagement?
Interruptions; talk-time ratio	are students learning to *listen*?
Hesitation before sensitive questions	does rehearsal reduce discomfort over sessions?
Realism, social presence, anxiety (survey)	is it believable and productive — or uncanny?

Do not over-claim. The sample is small and the semester is short. *"Twelve students, counterbalanced, with these effect sizes and confidence intervals"* is a credible result. *"Avatars improve learning"* is not. **A well-designed study with an honest negative result is the better deliverable** — and it will be graded that way.



The teal row is the genuinely novel contribution: whether a student lets the client finish talking may be a better measure of advising skill than anything the current rubric captures.

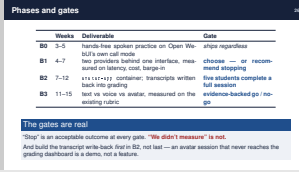
Phases and gates

	Weeks	Deliverable	Gate
B0	3–5	hands-free spoken practice on Open WebUI's own call mode	*ships regardless*
B1	4–7	two providers behind one interface, measured on latency, cost, barge-in	**choose — or recommend stopping**
B2	7–12	`avatar-app` container; transcripts written back into grading	**five students complete a full session**
B3	11–15	text vs voice vs avatar, measured on the existing rubric	**evidence-backed go / no-go**

The gates are real

"Stop" is an acceptable outcome at every gate. **"We didn't measure" is not.**

And build the transcript write-back *first* in B2, not last — an avatar session that never reaches the grading dashboard is a demo, not a feature.

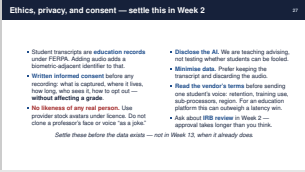


Make the kill switch explicit and unembarrassing. Research tracks that cannot fail are not research tracks.

Ethics, privacy, and consent — settle this in Week 2

- Student transcripts are **education records** under FERPA. Adding audio adds a biometric-adjacent identifier to that.
- **Written informed consent** before any recording: what is captured, where it lives, how long, who sees it, how to opt out — **without affecting a grade**.
- **No likeness of any real person.** Use provider stock avatars under licence. Do not clone a professor's face or voice "as a joke."
- **Disclose the AI.** We are teaching advising, not testing whether students can be fooled.
- **Minimise data.** Prefer keeping the transcript and discarding the audio.
- **Read the vendor's terms** before sending one student's voice: retention, training use, sub-processors, region. For an education platform this can outweigh a latency win.
- Ask about **IRB review** in Week 2 — approval takes longer than you think.

Settle these before the data exists — not in Week 13, when it already does.



Do not rush this slide. For a university deployment it is the one that determines whether the project is allowed to run at all.

Execution

Timeline, roles, and risk

How the two tracks stay in balance

The semester

Wk	Starts	Workstream A — platform	Workstream B — avatars
1	Aug 24	set up; run the stack; read the handoff	scope the landscape
2	Aug 31	**CI skeleton + fork guard + gitleaks**	requirements; **consent and IRB questions**
3	Sep 7	snapshot production; restore into test	**B0: voice session end to end**
4	Sep 14	test stack to v0.9.x; migrate the filter	provider shortlist; cost model
5	Sep 21	v0.10.x on test	**B0 demo**; latency baseline measured
6	Sep 28	v0.11.x on test; docs re-shot	provider #1 in a bare harness
7	Oct 5	**production upgrade window**	provider #2; head-to-head
8	Oct 12	hosted dashboard: container, proxy, TLS	**Gate 1 — pick, or stop**
9	Oct 19	dashboard auth; professor walkthrough	companion app skeleton
10	Oct 26	OTel + alerting live	avatar joins the session; barge-in works
11	Nov 2	scoring consolidation + tests	transcript write-back
12	Nov 9	RAG decision executed	**Gate 2 — five real sessions**
13	Nov 16	performance and cost review	evaluation sessions run
14	Nov 23	*(short week)* runbooks and documentation	data analysed
15	Nov 30	freeze; handoff written	recommendation drafted
16	Dec 7	**final presentation and handoff**	

Point at weeks 2, 7, and 8. A and B are deliberately interleaved so neither starves. The upgrade lands before the avatar work gets heavy.

Who owns what

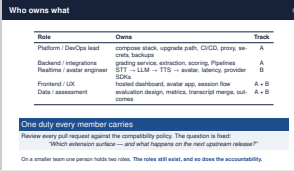
Role	Owns	Track
Platform / DevOps lead	compose stack, upgrade path, CI/CD, proxy, secrets, backups	A
Backend / integrations	grading service, extraction, scoring, Pipelines	A
Realtime / avatar engineer	STT $\rightarrow$ LLM $\rightarrow$ TTS $\rightarrow$ avatar, latency, provider SDKs	B
Frontend / UX	hosted dashboard, avatar app, session flow	A + B
Data / assessment	evaluation design, metrics, transcript merge, outcomes	A + B

One duty every member carries

Review every pull request against the compatibility policy. The question is fixed:

“Which extension surface — and what happens on the next upstream release?”

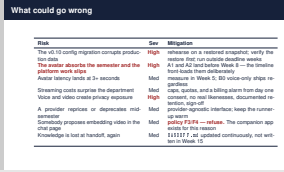
On a smaller team one person holds two roles. **The roles still exist, and so does the accountability.**



Roles are about accountability, not headcount. The shared review duty is what keeps the policy alive when the deadline pressure arrives.

What could go wrong

Risk	Sev	Mitigation
The v0.10 config migration corrupts production data	**High**	rehearse on a restored snapshot; verify the restore *first*; run outside deadline weeks
The avatar absorbs the semester and the platform work slips	**High**	A1 and A2 land before Week 8 — the timeline front-loads them deliberately
Avatar latency lands at 3+ seconds	Med	measure in Week 5; B0 voice-only ships regardless
Streaming costs surprise the department	Med	caps, quotas, and a billing alarm from day one
Voice and video create privacy exposure	**High**	consent, no real likenesses, documented retention, sign-off
A provider reprices or deprecates mid-semester	Med	provider-agnostic interface; keep the runner-up warm
Somebody proposes embedding video in the chat page	Med	**policy F3/F4 — refuse.** The companion app exists for this reason
Knowledge is lost at handoff, again	Med	`HANDOFF.md` updated continuously, not written in Week 15

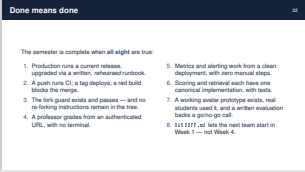


Row two is the one to dwell on. The exciting track will try to eat the boring track, and the boring track is what the course actually depends on.

Done means done

The semester is complete when **all eight** are true:

1. Production runs a current release, upgraded via a written, *rehearsed* runbook.
2. A push runs CI; a tag deploys; a red build blocks the merge.
3. The fork guard exists and passes — and no re-forking instructions remain in the tree.
4. A professor grades from an authenticated URL, with no terminal.
5. Metrics and alerting work from a clean deployment, with zero manual steps.
6. Scoring and retrieval each have one canonical implementation, with tests.
7. A working avatar prototype exists, real students used it, and a written evaluation backs a go/no-go call.
8. `HANDOFF.md` lets the next team start in Week 1 — not Week 4.



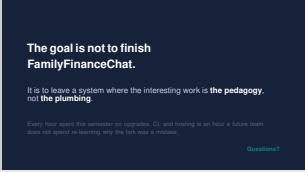
Read these as acceptance criteria, not aspirations. Every one is verifiable by someone who was not in the room.

The goal is not to finish FamilyFinanceChat.

It is to leave a system where the interesting work is **the pedagogy**, not **the plumbing**.

Every hour spent this semester on upgrades, CI, and hosting is an hour a future team does not spend re-learning why the fork was a mistake.

Questions?



Close on purpose, not on tasks. Then open the floor – expect questions on avatar cost, on privacy, and on whether the upgrade can wait. It cannot.